DSPham_Notes_R1 – A DSP audio processor aimed at radio amateurs

https://github.com/grahamwhaley/DSPham/blob/master/README.md

Introduction

The purpose of this document is to capture some additional images, notes and information that may be helpful to others constructing, using or customizing the DSPham project accessible via the hyperlink above. A good deal of this content comes directly from Graham Whaley's README.md file, email exchanges with Graham, DSPham source code, and additional background reading either pointed to via the README.md file, references within the source code, or other papers and texts on DSP processing of speech signals.

Construction presented in Images with descriptions

In this section, I present some images related to the construction of the DSPham unit.

NOTE: This document refers to the Teensy V4.1 board and compatible "Audio Shield".
Verify pins if using different versions of the Teensy product line with their compatible audio shields.

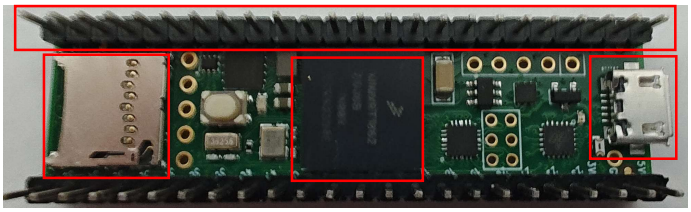


Figure 1. The Teensy 4.1 Controller board with micro- USB connector on the right, and SD card slot on the left, ARM-Cortex M7 processor in the middle. Note the installation of the header pins.

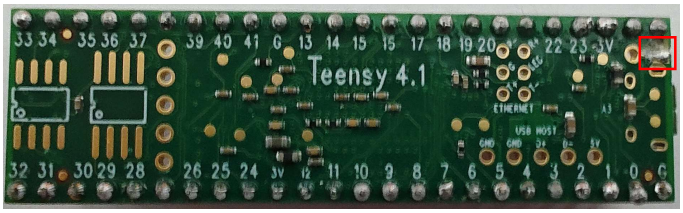
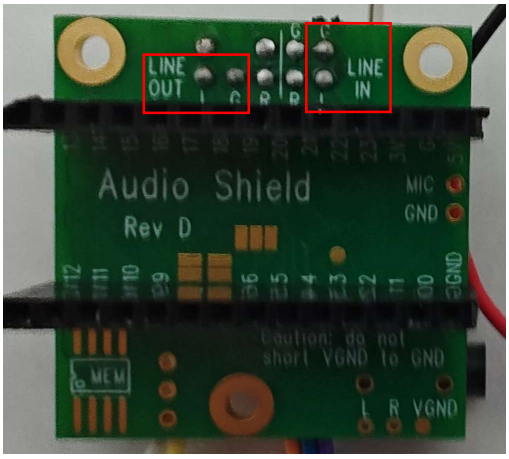


Figure 2. The underside of the Teensy 4.1 Controller board showing where the trace between two solder pads is cut out using a box-cutter, so that supplemental power does not conflict with the 5V from the micro-USB connection.

Figure 3. The underside of the audio shield board. Note the installation of the connectors which will mate with the headers soldered to the Teensy 4.1 board.

Of particular interest are the LINE OUT pins, and LINE IN pin solder points. In each case, one pin is labeled "L" for left-channel, and the other "G" for ground.



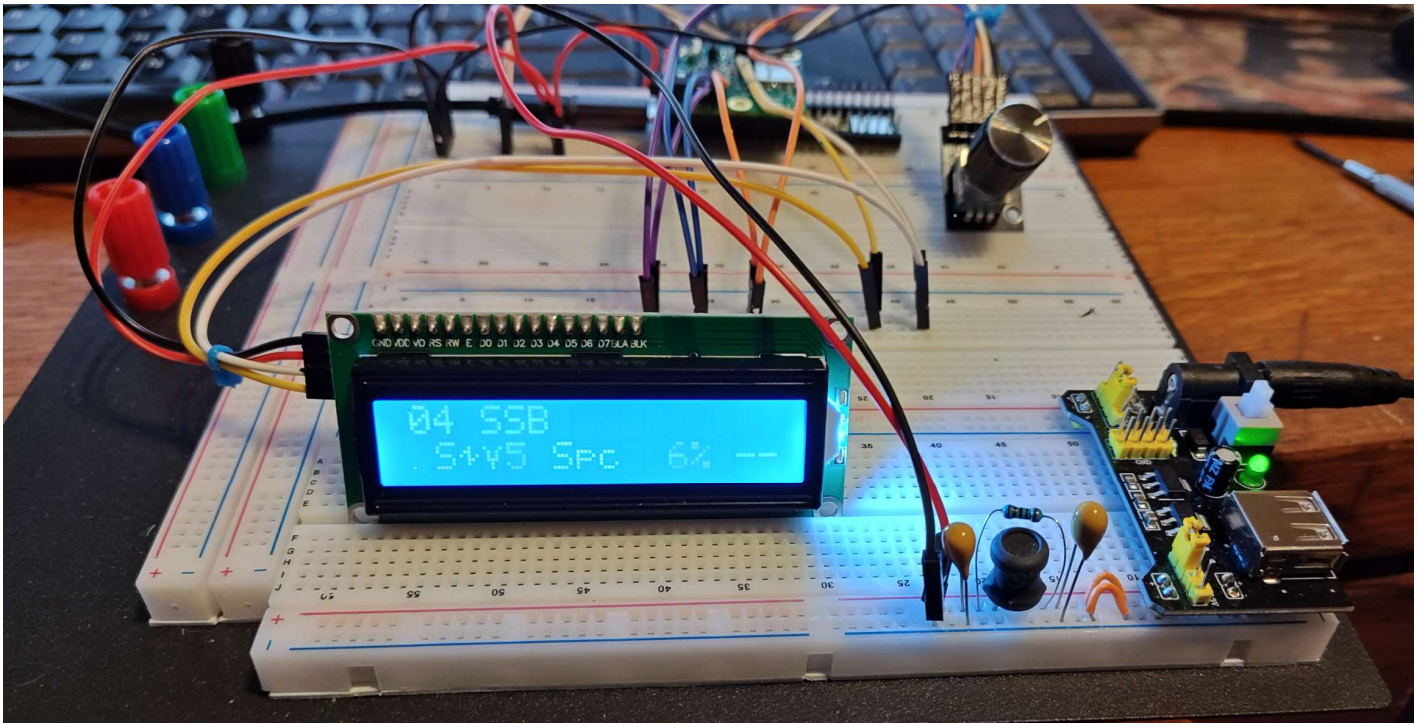


Figure 4. Breadboard construction

- Various elements of the design were connected using a breadboard and Dupont connectors.
- Lower-right: A power module board used for converting a range of input voltages: 7.0 to 12.0VDC to 3.0 and 5VDC. Note on this power supply board, the 3VDC output is not used.
- Not shown: The 12VDC AC Adapter supplying power to the power module.
- Also Lower-right: a small power filter consisting of 2 capacitors, one inductor, and one resistor. Not really needed. See the end of this document for more information.
- Foreground: The 2line x 16 character LCD. Not shown, mounted behind the LCD is a 1602 I2C controller.
- Background Upper-right: KY-040 Encoder module.
- Background Middle: Teensy 4.1 board with Audio shield.

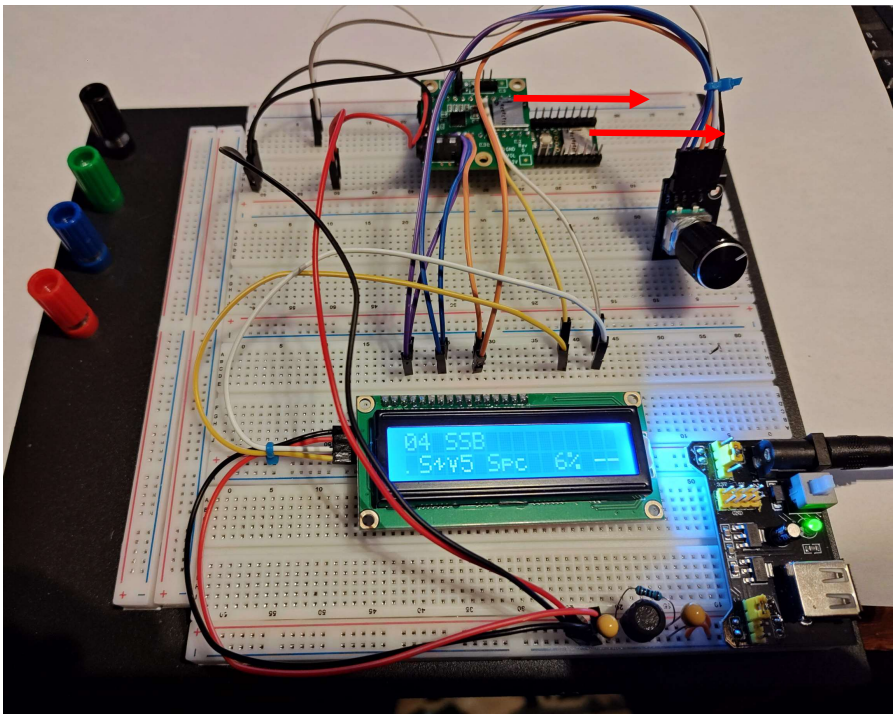


Figure 5. Breadboard construction view from above

Note the wires connecting to the audio shield, and also the orientation of the Teensy 4.1 and audio shield and their respective SD card slots pointing in the same direction as highlighted by the arrows.

Not connected in this image is the USB connection to the PC running the Arduino IDE for programming the Teensy.

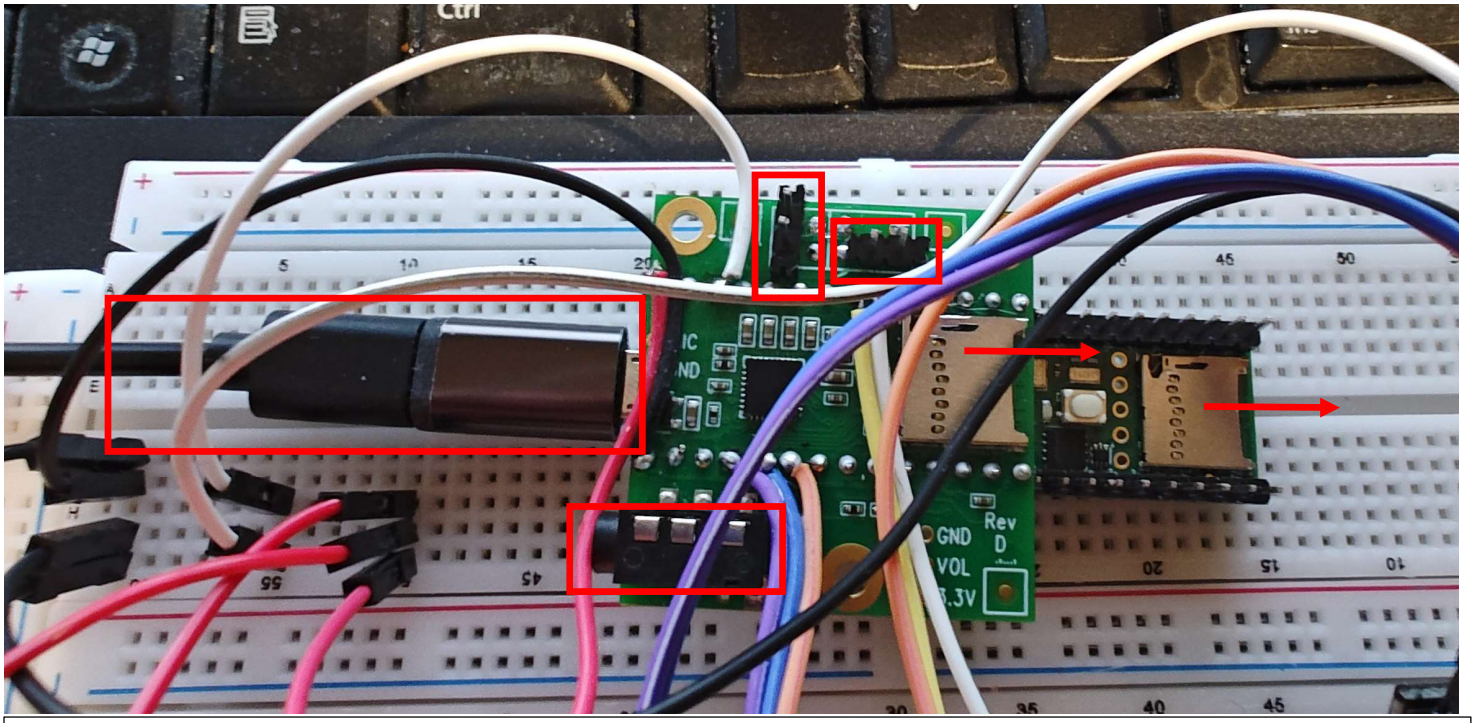


Figure 6. Closeup of the Teensy 4.1 board (on the bottom), and audio shield (on the top).

Lower left: The audio shield's stereo Headphone jack. Only the left-channel is used in the project. FWIW: I have been using a set of SONY MDR-V250 headphones (not shown plugged in here in this image). Specs of the headphones used: 500mW power handling, stereo with in-line switch for mono listening; freq. response: 18.0Hz to 22.0kHz, impedance: 24ohms.

Left: USB Connection is made using a USB-A to USB-C, then a USB-C to micro-USB adapter as I had no USB-A to micro-USB cables available.

Upper Middle: two headers soldered to the audio shield. The one on the left, perpendicular to the USB connection is Left-channel LINE IN. (Only one of the stereo channels of the Teensy and audio shield are used). On the right, parallel to the USB connection is Left-Channel LINE OUT.

Note: Repeating - when the audio shield is mounted atop the Teensy board (left-aligned on the header pins), the SD card slots of both the Teensy and audio shield point in the same direction as shown by the arrows!

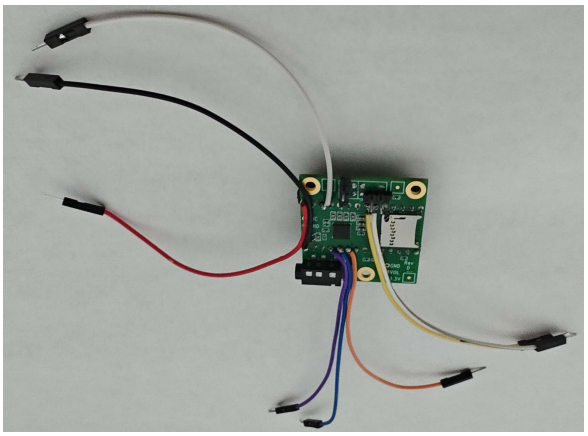


Figure 7. The audio board (I call this the "spider" for obvious reasons) with Dupont wires are directly soldered on the top of the audio shield to the pins of interest.

- Note the orientation of the audio shield and the pins shown, also shown in **Figure 8.**
- Construction tip: To provide some strain-relief, some wires have been super-glued together, and then to the audio shield.
- Red and Black: **Vin** 5VDC and **GND** respectively.
- White: **3.3VDC** (generated by the Teensy and used by the KY-040 rotary encoder).
- Purple, Blue, and Orange: **OUT2**, **LRCLK2**, and **BCLK2** respectively, also supporting the rotary encoder.
- Yellow and White/wBlack stripe: **SCL** and **SDA** supporting the 2line x 16 character LCD.

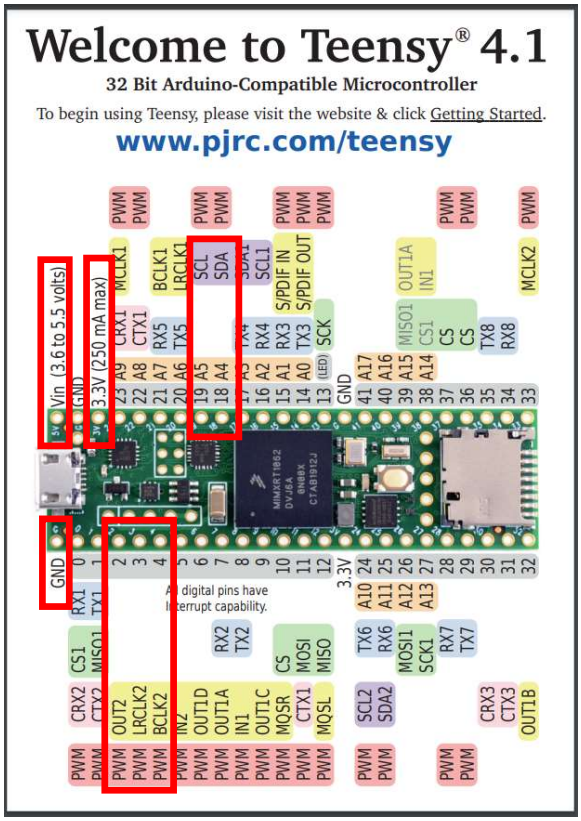
Figure 8. Teensy 4.1 documentation from the web-page shown.

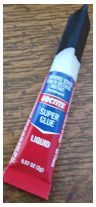
Vin and **GND** – power

3.3V – 3.3VDC supporting the rotary encoder

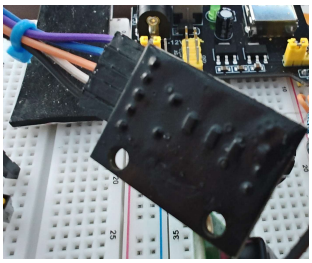
SCL and **SDA** feeding the LCD

OUT2, **LCCLK2**, **BCLK2 –** note the **OUT2** pin label implies it is an output pin, when in actuality, the digital pins default to inputs and can be used for input or output, with active high, or active low states. See Teensy documentation for more details.









Figures 9, 10, 11. More construction tips: Dupont connectors at the LCD and rotary encoder are grouped using cable ties, and then super-glued together to create a "connector".

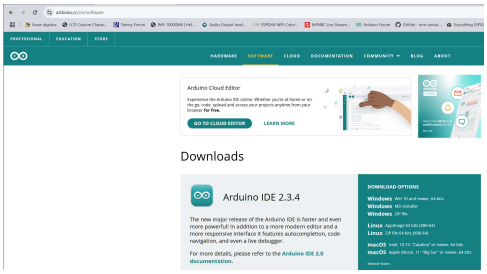
To prevent accidental shorting, a liquid electrical tape product can be applied to the connectors and then to the underside of modules if desired.

Both products are available at home improvement and hardware stores, Amazon, etc.

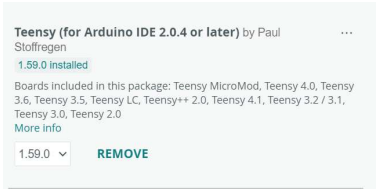
Setting up the IDE and libraries to compile and upload program

(This section will require me to go through the process on a computer that I haven't been using previously to verify steps and order… Therefore assume it may change to some degree.)

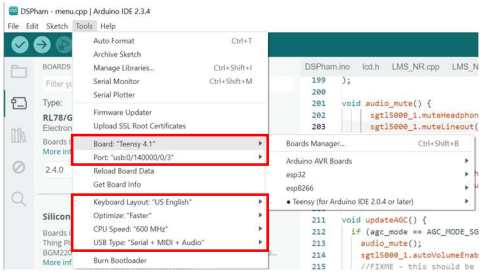
- Review the documentation in the GitHub repository README.md file using the hyperlink at the beginning of this document.
- Download the Arduino IDE at: http://arduino.cc/en/software



- In the IDE select Tools | Boards |Boards Manager and add the Teensy 4.1 board by selecting:



- Connect the Teensy board to a USB port on the PC
- Verify you've configured the Teensy 4.x board for Teensy 4.x, Serial+MIDI+Audio setup with the following settings (the actual usb port may be different than that shown here):



- Verify communication to the Teensy by compiling and uploading a sketch by selecting Examples | _Teensy |Tutorial1 | blink, or equivalently copying and pasting the following into the IDE:

```
/* LED Blink, Teensyduino Tutorial #1
   http://www.pjrc.com/teensy/tutorial.html

   This example code is in the public domain.
*/

// Teensy 2.0 has the LED on pin 11
// Teensy++ 2.0 has the LED on pin 6
// Teensy 3.x / Teensy LC have the LED on pin 13
const int ledPin = 13;

// the setup() method runs once, when the sketch starts

void setup() {
  // initialize the digital pin as an output.
  pinMode(ledPin, OUTPUT);
}

// the loop() methor runs over and over again,
// as long as the board has power

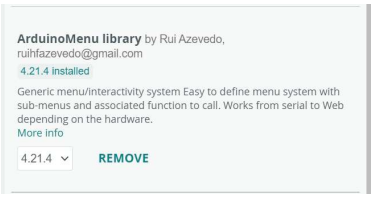
void loop() {
  digitalWrite(ledPin, HIGH);   // set the LED on
  delay(1000);                  // wait for a second
  digitalWrite(ledPin, LOW);    // set the LED off
  delay(1000);                  // wait for a second
}
```

- At the end of the upload the Teensy may reboot and start running the sketch, but if not, when a window pops up on the PC indicating to press the RESET button on the Teensy, do that.
- Assuming the program runs and the LED on the Teensy blinks:
    - Skip adding the Grove RGB LCD library.

- Add the **LiquidCrystal_PCF8574** library using the library manager. The PCF8574 is the module attached to the back of the LCD that communicates via the I2C protocol and connection.



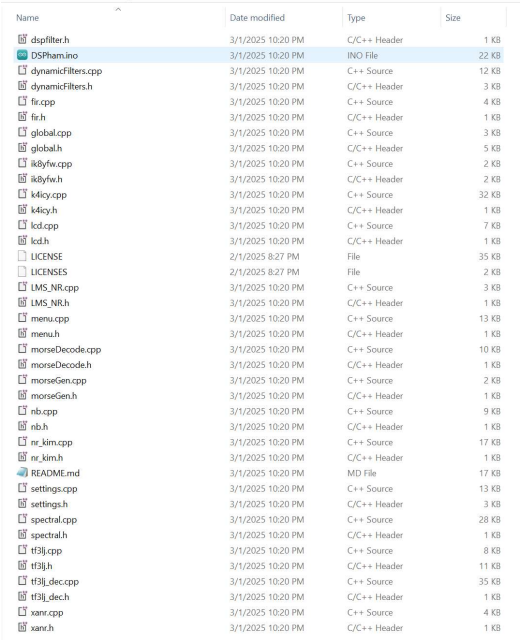
- Add the **ArduinoMenu** library by RuiAzevedo via the library manager.



- Add the **Click Encoder** Library.
  - Go to this location: https://github.com/soligen2010/encoder
  - Click on the Green Code button and download zip file, but don't unzip it.
  - Install the library by selecting menu item Sketch | Include Library | Add .ZIP Library… and point to the downloaded .zip file.
- Download the Code for DSPham from this location: https://github.com/grahamwhaley/DSPham
- Place the .zip file in a directory you intend to build the project from and unzip the file.
- Likely a DSPham-master directory will be created with the following 4 sub-directories:



- Copy any source files from the DSPham project into the DSPham directory containing the **DSPham.ino** sketch. The results should look something like the following:



- Now follow the steps outlined near the end of the document which starts with:

  "The Grove RGB LCD can be replaced by a much less expensive 2line x 16character LCD that is monochromatic"

  Note: I found it informative and instructive to look at the differences between Graham's version with the RGB LCD, and that of the robaol's pull at the hyperlink provided, and to make the changes manually, although, if one wishes, one could download robaol's pull and compile and upload that.

Using the Encoder to navigate the menu system

- Clockwise rotation, advances deeper into the menu hierarchy, or increases a parameter
- Counter-clockwise, moves back up a level within the menu hierarchy, or decreases a parameter
- Select – depressing / clicking the encoder makes a selection, i.e. entering into a menu or sub-menu
    - When ":" is displayed, cw or ccw rotation of the control will increment or decrement the setting's value
    - When ">" is displayed ???? what/why ????
    - When " " is displayed, cw or ccw rotation of the control moves down or up in the menu / sub-menu hierarchy of options and settings

The following top-level menus are available

Volume – adjusts the volume of the Headphones output {0.0 -> 1.0}. The headphones socket is located on the Audio Adapter board mated with the Teensy controller. Adjustments to the Volume setting has no impact to the line-out signal from the Audio Adapter.

NR menu – allows selection of a noise reduction mode, from a number of different noise reduction algorithms:

NR Mode:

Off – no noise reduction is performed

LMS – Least Means Squared – a form of "adaptive filtering" where the coefficients of a digital filter are varied over time to minimize the error so that the output signal sent to the headphones and line-out connections on the audio adapter board most closely matches that of the input applied to line-in minus the noise corrupting the signal. An important assumption is that the target signal is independent of the noise which corrupts it for best performance. The LMS approach was originally developed by B. Widrow in the 1970s.

Kim – see source code: nr_kim.cpp, a method of processing noisy speech minimum ratio [NSMR] where the speech with noise is processed by first calculating a STFT (short-time Fourier transform) over L-frames of acoustic data to form E(k, l) through averaging; where k is a frequency bin index, and l is a frame number. Then a minimum function M(k,l) is obtained by taking the minimum at each frequency of the previous frame's min statistic and the current averaged STFT M(k,l) = min{M(k,l-1), E(k,l)}. Then a ratio statistic T(k,l) = |X(k,l)|/M(k,l) is computed. A weighted noise estimate is formed, then a spectral set of gain factors G(k,l) is computed from that, and the gains are smoothed by Gsmth(k,l) = alpha*Gsmth(k,l-1) + (1-alpha)*Gsmth(k,l); where alpha is a time-smoothing gain factor {0<alpha<1}. Smoothing is increased as alpha is increased; one might think of alpha as a frequency bin gain or smoothing factor. Then an additional local gain factor is applied to neighboring frames using the previous frame, present frame, and future frame gains regulated by another smoothing constant: beta {0 < beta <= 0.5}. The higher the value of beta, the more the previous and future frame's gain factors influence the processing of the current frame. One can think of beta as an inter-frame gain or smoothing factor. Lastly the algorithm takes the Inverse Fourier Transform (IFFT) of the Gain / spectrally-smoothed magnitude multiplied by the FFT of the input signal and the input signal's phase, to produce an estimate of the noise-reduced input.

See the paper: Kim, H.-G. & D. Ruwisch (2002): Speech enhancement in non-stationary noise environments. – 7th International Conference on Spoken Language Processing [ICSLP 2002]. – ISCA Archive (http://www.isca-speech.org/archive), specifically: https://www.isca-archive.org/icslp_2002/kim02g_icslp.pdf

fnr – exponential smoothing moving filter, see source code: ik8yfw.cpp

Giuseppe Callipo – IK8YFW - ik8yfw@libero.it

See: www.github.com/gcallipo/RadioDSP-Stm32f103 and

https://en.wikipedia.org/wiki/Moving_average

fnrA – average smoothing moving filter, see source code: ik8yfw.cpp; the larger the NR tweaks setting, the more samples of the input signal that are averaged together.

Spectral – spectral subtraction, see source code: spectral.cpp

The spectral subtraction method of removing noise attempts to model (via FFT) the underlying noise magnitude spectrum from the source audio. The noise spectrum is estimated during non-speech periods determined by a speech "activity detector". Then via subtraction, this noise spectral estimate is taken from the source audio to obtain an spectral estimate of the "clean speech", and any negative magnitudes resulting from the subtraction are clipped at zero. The result is then converted back into the time-domain by IFFT with the potential merging of the result back into the previous frame of noisy speech.

See: Boll, S., ``Suppression of Acoustic noise in Speech Using Spectral Subtraction'', IEEE Trans. Acoustics, Speech, and Signal Processing ASSP 27: 113-120, 1979
https://course.ece.cmu.edu/~ece491/homework/Boll79.pdf

Also see: Romain et. all 2009, Schmitt et all 2002, Gerkmann and Hendriks 2002, and Yao et all 2016 DD4WH and Michael DL2FW, Nov. 2017

see: https://github.com/df8oe/UHSDR/wiki/Noise-reduction

LLMS – Leaky Least Means Squared algorithm, is a leaky version of LMS in that the algorithm can adapt to the changing noise, through the adjustment of filter coefficients as in the LMS case. In addition, some "leakage factor" {0,1} is used to "forget" the coefficients of the filter which are generally iteratively determined, thereby better adapting to changing noise conditions.

See: "Experimental evaluation of leaky-least-mean-square algorithms for active noise reduction in communication headsets", David A. Cartes, Laura R. Ray, Robert D. Collier, Journal of Acoustic Society of America, April 2002, pgs 1758-1771.

NR tweaks sub-menu (default values presented in [])

LMS Strength [5] can be set from 0 to 40
Kim str (Kim strength) [1.0] can be set from 0.80 to 1.0
Kim PSI [3.0] can be set from 1.0 to 6.0
Kim alpha [0.95] can be set from 0.70 to 0.99
Kim beta [0.85] can be set from 0.10 to 0.99

???? would be nice to have an approachable description of the significance of these settings ????

FNR level [2] can be set to 1, 2, or 3; 1 is the min setting, resulting in a multiplier of 0.5, 2 resulting in a multiplier of 0.3, and a setting of 3 which is the max results in a multiplier of 0.1

FNRA level [5] can be set to 4, 5, or 6; a setting of 4 results in the averaging of 8 data samples, a value of 5 results in the averaging of 16 data samples, and a value of 6 results in the averaging of 24 data samples from the source.

><Back – returns to a level above the current level in the menu hierarchy

Notch Mode sub-menu for selecting the notch mode on/off. The notch function reduces the presence of a dominant tones which may be corrupting the input signal.

On – enables the Notch filter.
Off – disables the Notch filter.

Blnkr Mode sub-menu for selecting the noise blanker mode on/off. The blanker function works to remove impulse noise.

On – a means of removing impulses from the source audio, and replacing them with a linear-predicted value based on prior input samples. First a set of linear prediction coefficients model the vocal tract of the speech, then inverse filtering the source with this model to remove the speech (but not the noise), detect impulses by thresholding, then replacing those samples in the original source within the vicinity of the impulses using forward and backwards prediction. LPC coefficients by Levinson-Durbin. See the source file: nb.cpp.

Off – turns the function off.

><Back – returns to a level above the current level in the menu hierarchy

Decoder menu

Dcdr Mode

OFF – The source audio is not processed to convert morse to text.

Morse – The 'morseduino' decoder by GI1MIC and WB7FHC (Budd Churchward)

K4ICY – The K4ICY decoder for Arduino, using a geometric mean for pulse categorization, a great description of the methodology employed is found here:
http://k4icy.com/cw_decoder.html

TF3LJ – Bayesian based decoder by Loftur E. Jonasson – TF3LJ / VE2AO
https://sites.google.com/site/lofturj/repository-for-recent-amateur-radio-projects and specifically:
https://sites.google.com/site/lofturj/14-machine-recognition-of-hand-sent-morse-code
and based on the following PDP-12 implementation, documentation found here:
https://apps.dtic.mil/sti/tr/pdf/AD0786492.pdf
The disadvantage of this algorithm is the excessive amount of time (on the order of 30sec) needed to model the statistics of the input signal and establish a lock on the decoding process using those statistics. When that "lock" on the input signal it's decoding is lost, it takes tens of seconds again to re-lock and begin to decode the input signal.

><Back – returns to a level above the current level in the menu hierarchy

Dcdr tweak

CW Freq [600] can be adjusted 1Hz – 1000Hz – the Rx pitch / frequency of the CW source

CW Tsh – [0.01] can be adjusted .01 – 1.0 – the CW threshold. The value of .01 is a rather low value. Too low of a threshold results in mis-identifying a "dit" or "dah" from a signal with noise. Too high a value may result in rejecting signal level changes corresponding to the beginning of a dit or dah.

CW cyc – [14] can be adjusted 1 – 100 – the CW cycles ???? is this the number of dit or dah statistic modelling passes before decoding starts ????

><Back – returns to a level above the current level in the menu hierarchy

Filter menu

Flt Md (filter mode, see source: filter.cpp and https://github.com/gi1mic/19Dollar-DSP-Filter)

PThru (pass through) – source is processed by a bandpass filter (BPF), 60Hz – 20kHz.

SSB – source processed by a BPF, 300Hz – 2700Hz

CW – BPF, 450Hz – 950Hz

AM – BPF, 50Hz – 11kHz

FM – BPF, 50Hz – 16kHz

><Back – returns to a level above the current level in the menu hierarchy

Filter tweaks

Frequency Low [0-20kHz]

Frequency High [0-20kHz]

><Back – returns to a level above the current level in the menu hierarchy

AGC menu

AGC Type

Off – No automatic gain control applied in generating the output signal

Track – attempt to track the value of the input signal's peak energy / mean energy

SG5K – use the AVL (automatic volume level, i.e. AGC) capabilities built into the SGTL5000 integrated

circuit on the audio adapter board, with programmed levels for threshold, attack and decay rates

><Back – returns to a level above the current level in the menu hierarchy

Settings menu

Edit Slot [0] – enter into the edit slot mode.

Slot Name [FM] – name the slot mneumonically.

Save Slot – save the changes to a slot.

Reset Slot – selectively reset a particular slot back to default.

Factory reset – all slots will be reset to defaults.

ScrnRGB [404040] specifying the background for the Grove RGB Backlight LCD.

Note: changing these values have no effect if the backlight of the LCD being used is monochromatic.

To do: document removing this from the Settings menu.

Ver – display the version of the program (as of this document, V1.3)

Discussion of "Slots"

There are 11 slots or storage locations for establishing preset configurations that can be recalled and used from non-volatile memory (i.e. memory which is persistent across circuit power cycles).

- The first 6 slots default to the following, but may be changed thru the menu.
- Modifications to the configurations can be made using the **Edit Slot** option.
- When **Save Slot** is selected, the setting changes become persistent across power-cycling of the DSPham module.
- Slot configurations can be individually restored to Factory Defaults (the values encoded in the source code and built and uploaded to the Teensy) via the **Reset Slot** menu option.
- A re-initialization of all the slots occurs if a **Factory Reset** menu option is selected. Note: there is currently no Confirm Yes/No? type choice presented to the user; the factory reset occurs immediately when clicking the encoder.

Default Slot Configurations (i.e. prior to any modification by the user and/or following a "Factory Reset")

- 0: FM - Wide band FM filter, notch and nb on, spectral NR.
- 1: AM - Wide band AM filter, notch and nb on, spectral NR.
- 2: OFF - Everything off (no processing).
- 3: SG5K - Everything off apart from hardware SG5k AGC.
- 4: SSB - Medium band SSB filter, notch and nb on, spectral NR.
- 5: CW - Narrow band CW filter, notch off, nb on. No NR. [K4ICY][7] cw decoder
- 6 – 10: These are initially "Empty" slots (following a Factory Reset) open to store other configurations besides the defaults, or modified
  default slots.

Description of the Contents of the 2 Line LCD Display

Line 1:

The first two characters indicate the program "slot", i.e. configuration that is active.

The second set of two, three, or four characters indicate the "slot type", i.e. the type of processing being applied to the supplied audio signal from the source when generating output.

However, if the unit is decoding CW, then line 1 presents a scrolling view of the output of the decoder.

Line 2:

The first character represents the input signal level, and also an indication if the source is exceeding the max line level defined to the chip on the audio processing board going from no signal to overload:

The second character represents the type of filter being employed ???

The third character indicates if Noise Blanker is enabled or '-' if it is not enabled

The fourth character indicates if Notch Filter is enabled or '-' if it is not enabled

The fifth character is associated with AGC

The sixth character is always blank

The seventh, eighth and nineth character represent the Noise Reduction Algorithm active or 'Off'

The tenth character is always blank

The eleventh, twelfth, and thirteenth characters represent CPU% utilization

Note: The program tracks the number of microseconds elapsed since controller boot up. This value wraps around approximately every 71minutes and 35seconds as it is a 32bit number. With respect to cpu usage, i.e. x%, the controller calculates the number of microseconds elapsed through each pass.

The fourteenth character represents CWTUNE during CW decode (i.e. how close is the Rx pitch/frequency to that defined to the decoding algorithm), the following being a customized character formed for display on the second line of the LCD:

Notes: if the received tone is pretty close to the value in settings (i.e. optimal), then will be displayed

If the tone is too low, will be displayed

If the tone is a little low, will be displayed

If the tone is too high, will be displayed

If the tone is a little high, will be displayed

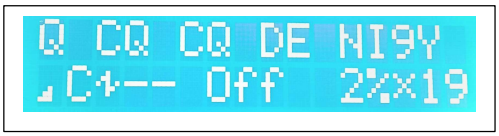
The fifteenth and sixteenth characters represent an estimate of the Words Per Minute (WPM) during CW decode; otherwise in other non-CW modes '- -' is displayed.

Some 2-Line Display Examples:

Slot 4 (named SSB) selected.
Audio input level good w/headroom
S = SSB Filter
Noise Blanker enabled
Notch Filter enabled
AGC Mode: **5** = SG5K
Noise Reduction mode: Spectral
6% CPU utilization

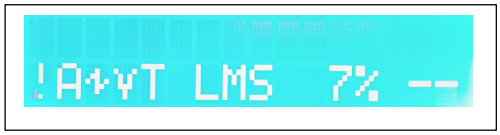


CW decode mode.
First line is the currently received message:
Second line (left to right)
Audio level good w/headroom



C = CW Filter
Noise Blanker enabled
Notch Filter disabled
AGC disabled
Noise Reduction 'Off'
2% CPU Utilization
Rx pitch matches the setting very closely
19 Words Per Minute

Input signal overload (!)
AM Filter
Noise Blanker enabled
Notch Filter enabled
AGC mode: **T**rack
Noise Reduction mode: LMS
7% CPU utilization



Customizations possible (not meant to be exhaustive)

- Defaults restored during a "Factory Reset" for various settings, stored in global.cpp (not an exhaustive list follows):
  - Default Noise Reduction (NR) mode: NR_MODE_SPECTRAL
  - Default Noise Reduction status: DECODER_OFF
  - Morse Code pitch/frequency: 600Hz
  - Morse Threshold: 0.01
  - Noise Blanker status: true
  - Notch Filter status: true
  - Default AGC mode: AGC_MODE_SG5K
    - Max gain, response time, hardlimit, threshold, attack and decay for the SGTL5000 integrated circuit
    - Default Headphone output volume: 0.5

- Depending on the encoder used, the direction of rotation can be manipulated, the clicks, and the indents per rotation. In menu.cpp
  #define encA 3
  #define encB 2
  #define encBtn 4
  ClickEncoder clickEncoder(encA,encB,encBtn,2); ← here steps per notch = 2. ((I modified this with a value of 4))
  ClickEncoderStream encStream(clickEncoder,1);

- The audio source line-in, and processed line-out levels are defined in DSPHam.ino
  sgtl5000_1.lineInLevel(7); //0.94v p-p
  sgtl5000_1.lineOutLevel(31); //1.16v p-p
  According to documentation for the SGTL5000, max line-levels can be as much as 3 volts peak-to-peak (p-p).

- The declarations of the 8 custom characters designed for display on the 16x2 LCD display are found in the source file: lcd.cpp.

- The Grove RGB LCD can be replaced by a much less expensive 2line x 16character LCD that is monochromatic. The changes are made to the following repository pull:
  https://github.com/robaol/DSPham/tree/OtherLCDs

and specifically for the diffs between robaol version (using the monochrome LCD) and grahamwaley project:
https://github.com/grahamwhaley/DSPham/compare/master...robaol:DSPham:OtherLCDs
Note that in the implementation on a 16x2 LCD and with the changes specified by robaol, I had to do two things:
First - add a call to lcd.setBacklight(HIGH) to turn the LCD's backlight on (additional mods in green).

```
load_colour();    //load lcd screen colour.
 //stf mod 2/16/2025 turn the backlight of the display I'm using on
 #ifdef USE_MATHERTEL_LCD
 lcd.setBacklight(HIGH);
 #endif
morseInit();
```

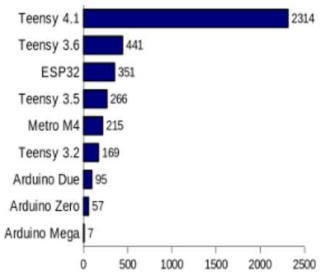
Second - I had to modify LCD_ROWS and LCD_COLS in file: lcd.h:

```
#ifdef USE_MATHERTEL_LCD
#include "LiquidCrystal_PCF8574.h"
#define LcdType LiquidCrystal_PCF8574
#define LCD_ROWS  (2)     //stf if the display had 4 rows of 20 characters these would be 4 and 20 respectively
#define LCD_COLS  (16)
#endif
```

Specifications and additional information

Teensy 4.1 Development board: https://www.pjrc.com/store/teensy41.html

- ARM Cortex-M7 at 600 MHz
- Float point math unit, 64 & 32 bits
- Coremark (Benchmark score) of 2314; a measure of the number of times per second a processor can perform a mix of common tasks, including linked-list management, matrix multiply, and state machine operations.



- Support for an audio processing library: https://www.pjrc.com/teensy/td_libs_Audio.html
  with DSP extension instructions to implement filters, and Fast Fourier Transforms (FFTs).

Teensy Audio Adapter board: https://www.pjrc.com/store/teensy3_audio.html
Specifics of the SGTL5000 integrated circuit: https://www.pjrc.com/teensy/SGTL5000.pdf

The Audio Adapter board based on the SGTL5000 chip can process 16bit stereo samples at 44.1kHz sample rate (effectively considered CD-quality). However, the DSPham program decimates this by a factor of 4, for effectively handling band-limited signals of 0-5.5kHz (where the Nyquist frequency is 11kHz).
Notes:

Most of the important information contained within human speech exists between 0-4.0kHz
Historically the bandwidth of analog telephone exchanges is in the range of 300Hz-3.4kHz.
Human hearing is generally limited to the range: 20Hz – 20kHz.
https://en.wikipedia.org/wiki/Voice_frequency

Voltage and current consumed by a 4.1 Teensy controller, including the audio board, encoder, and Grove RGB LCD display:

Vsupply: 5.0V
Adraw: 210mA (approximately)

Current draw consumed by the same configuration but with the following LCD replacing the Grove RGB LCD
https://www.amazon.com/dp/B0BWTFN9WF?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1
Adraw: 150mA @ 5.0V

Grove LCD Backlight
Docs: https://wiki.seeedstudio.com/Grove-LCD_RGB_Backlight/
Voltage: 5V, Current: <60mA
LCD I2C address: 0x3E, RGB I2C address: 0x62

PCF8574 I2C adapter driven 16x2 LCD
Docs: https://www.mathertel.de/Arduino/LiquidCrystal_PCF8574.aspx
https://github.com/mathertel/LiquidCrystal_PCF8574

KY-040 Rotary Encoder
Doc: https://www.handsontec.com/dataspecs/module/Rotary%20Encoder.pdf

Miscellaneous:

ggmorse source code for decoding CW. https://github.com/ggerganov/ggmorse
km1ndy Arduino morse decoder:
https://km1ndy.com/easy-step-by-step-diy-arduino-morse-code-decoder-for-cw-ham-radio-operators/
CW Decoder by Hjalmar Skovholm, 0Z1JHM located here:
https://web.archive.org/web/20191029193600/http://skovholm.com/cwdecoder
and specifically the source code here:
https://web.archive.org/web/20191003205415/http://www.skovholm.com/decoder11.ino
KC2UEZ modifications to K2JJI's modifications of 0Z1JHM source located here:
https://web.archive.org/web/20191028201550/http://k2jji.org/2014/09/18/arduino-base-cw-decoder/

Revision History

R_	2/07/2025	Initial Revision
R0	2/16/2025	Additional Notes Re: Use of a PCF8574-based I2C 16x2 White on Blue Backlit LCD Additional code to turn on the LCD backlight
R1	2/23/2025	Additional descriptions re: the different algorithms
R2	3/02/2025	Construction images and descriptions. Additional content.